

Advanced Algorithms and Data Structures

BSc Computing



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Hashing Algorithms Assignment

Oisin Gibson

L00172671

Declaration

- 1) I have accurately identified and included the sources of all facts, ideas, opinions, and viewpoints from others in the assignment references. All direct quotations, paraphrasing, and discussions of ideas from books, journal articles, internet sources, course materials, or any other sources used are properly acknowledged and cited in the assignment references.
- 2) I have not used unauthorised artificial intelligence tools or aids.
- 3) I understand and am compliant with ATU's policy and procedures regarding Academic Integrity and I am aware of the consequences of any violations.
- 4) I have followed the referencing guidelines recommended in the assignment instructions and / or programme documentation.
- 5) I understand that not adhering to the word limits on each section will result in penalties.
- 6) By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other programme of study.
- 7) By signing this form or by submitting material for assessment online I confirm that I have read and understood AQAE022 Academic Integrity Policy.

I confirm I agree to this declaration. ☒

Print Signature: Oisin Gibson

Date: 29/12/2025

Lecturer's Name: Ruth Lennon
Assessment Title: Hashing Algorithms Assignment
Work to be submitted to: Ruth Lennon
Date for submission of work: 29/12/2025
Place and time for submitting work: BlackBoard/GitHub

To be completed by the student	
Student's Name: Oisin Gibson	
Class: BSC Computing	
Subject/Module: Advanced Algorithms and Data Structures	
Word Count (where applicable): 1422	
I confirm that the work submitted has been produced solely through my own efforts.	
Student's signature: Oisin Gibson	Date: 29/12/2025

Notes	
Penalties: The total marks available for an assessment is reduced by 15% for work submitted up to one week late. The total marks available are reduced by 30% for work up to two weeks late. Assessment work received more than two weeks late will receive a mark of zero. Continuous Assessment: For students repeating an examination, marks awarded for continuous assessment shall normally be carried forward from the original examination to the repeat examination. Declaration: I declare that this work is entirely my own and does not contain the words or ideas of someone else, whether published or not, without specific acknowledgement by relevant referencing. I have read and understood the ATU Donegal Plagiarism Policy on the "Student & Academic Policies" section of the ATU Donegal Website and understand plagiarism to include: Direct copying of text, images and other materials (electronic or otherwise) from a book, article, fellow student's essay, handout, web page or other source without proper acknowledgement. Claiming individual ideas derived from a book, article etc. as one's own and incorporating them into one's work without acknowledging the source of these ideas. Overly depending on the work of one or more other sources without proper acknowledgement of the source, by constructing an essay, project etc., extracting large sections of text from another source and merely linking these together with a few of one's own sentences. I understand that it is my responsibility to familiarise myself with and to follow the Institute's Assessment Regulations. I acknowledge that Incidents of alleged plagiarism and cheating are dealt with in accordance with the Institute's Assessment Regulations and that penalties will be applied if I breach this policy.	
Signed: Oisin Gibson	
Date: 29/12/2025	

Contents

1	Question 1	5
1.1	Hash Function	5
1.2	Hash Table	5
1.3	Collision in a hash table	5
1.4	Load factor	6
1.5	Open addressing	6
2	Question 2	6
2.1	Discuss how hash-based data structures differ from tree-based data structures.	6
2.2	What are the advantages and disadvantages of each in terms of time complexity for operations such as insert, delete, and search?	7
2.2.1	Hash-Based data structures	7
2.2.2	Tree-Based Structures	8
3	Question 5 - Briefly describe how Question 4 can be updated to support concurrency	8
4	References	10
5	Appendix	11

1 Question 1

1.1 Hash Function

A hash function is considered a fundamental concept in computer science. It functions by creating a map from an inputted key to an index in a hash table. It underlines various critical applications such as data storage, retrieval, and cryptography.

They accept the input of any size and then produce a fixed-length string of characters which is known as a hash value or digest. The transformation is designed to be only one-way, which makes backing tracking next to impossible. (*Hash Functions and Types of Hash functions - GeeksforGeeks*, 2025).

Several key properties of hash functions are:

- **Determinism:** Where the same input will always produce the same output.
- **Collision resistance:** The probability that two different inputs will produce the same output is highly unlikely.

1.2 Hash Table

A hash table is a data structure which stores information as key-value pairs. To conclude where each value is stored in the table, a hash function will compute an index based on the key. The index will specify the location in the underlying array where the corresponding value is placed.

Due to this design features such as fast insertion, deletion, and searching can be completed with an average time of $O(1)$. Hash tables can often be found under different names such as associative arrays or dictionaries. (*Hash Table Data Structure*, 2025).

1.3 Collision in a hash table

Collision in a hash table occurs when the hash function maps two different keys to the same index in the hash table. Conflict is created as an index should really only hold one value. This makes it necessary for collision resolution techniques to be implemented to manage the situation. Collision may occur due to a limited table size or a poorly designed hash function. ("Hash Collisions Explained," no date).

1.4 Load factor

The load factor is the measure of how full a hash table is. It defined the ratio between the number of stored elements and the number of slots available, sometimes called buckets in the table. The formula used to calculate the load factor is as follows:

$$\text{Load Factor} = \frac{\text{Number of Elements}}{\text{Number of Buckets}}$$

Equation 1 Load Factor

A high load factor will increase the chances of collisions, which degrade the performance as multiple keys may end up mapping to the same index. While a low load factor will result in wasted memory usage. Most hash tables will implement automatic resizing, or rehashing, to ensure neither high nor low load factors occur. (*Load factor in hash table*, no date).

1.5 Open addressing

Open addressing is a technique used to resolve collisions in a hash table. It works by storing all the elements directly within the hash table itself. If a collision does occur, a probing sequence is used to find the next available slot for the new key. Some common methods of probing include:

- Linear – checks consecutive slots
- Quadratic – uses squared increments
- Double hashing – applies a second hash function to determine the step size

The process of probing works by first calculating an initial index. If the slot is occupied, then the probe will continue to the next until an empty slot is found. This approach can at times suffer issues if the load factor becomes high. (*Open addressing | Study Glance*, no date).

2 Question 2

2.1 Discuss how hash-based data structures differ from tree-based data structures.

Hash-based data structures differ from tree-based data structures in both their performance and in their functionality. A hash table uses a hash function to map keys to array indices. This allows for average-case constant-time operations for insertion,

deletion, and search. The performance may degrade to linear time if collisions are poorly handled. (*Advantages of BST over Hash Table*, 2025). With tree-based data structures however, this process differs. Balanced trees guarantee logarithmic-time operations by maintaining tree balance. This ensures that there is predictable performance regardless of the input distribution. (*Binary search tree vs. hash tables*, no date). Hash tables also do not have any inherent order among the keys. This makes them ill-suited to tasks that need data to be sorted or range queries. In contrast to this balanced tree naturally maintain order which means this support efficient traversals and range queries. In conclusion, hash tables are more often chosen for fast lookups and unordered data management. While with balanced trees are chosen when ordered data, hierarchical relationships, or deterministic performance assurances are needed.

2.2 What are the advantages and disadvantages of each in terms of time complexity for operations such as insert, delete, and search?

2.2.1 Hash-Based data structures

2.2.1.1 Advantages

Hash tables are known for their extremely fast average-case performance for insert, delete, and search functions. This is typically $O(1)$ as they implement a hash function which computes the index key directly. This makes them perfect when rapid lookups are required, like catching, symbol tables, and database indexing. The efficiency of hash tables is not dependent on the dataset size under ideal hashing conditions. This means that they scale well for large volumes of data. (*Advantages of BST over Hash Table*, 2025).

2.2.1.1.1 Disadvantages

The main disadvantage of a hash table is that the performance relies on the quality of the collision resolution plan and the hash function. If many keys hash to the same index and a collision occurs, the operational functionality can degrade to $O(n)$. This makes them unreliable for system that need guaranteed performance. Hash tables also do not maintain any inherent order among its keys. This limits usefulness for tasks that involve range queries or ordered traversals. (*Binary search tree vs. hash tables*, no date).

2.2.2 Tree-Based Structures

2.2.2.1 Advantages

Tree-based structures like balanced trees provide strong guarantees in terms of fundamental operations. (*Balanced Binary Tree definition & meaning in DSA*, 2025). When the tree is balanced, the height is maintained at $O(\log n)$. This ensures that functions like search, insertion, and deletion all function in $O(\log n)$ time. (*DSA AVL Trees*, no date). This is a key advantage over that of linear structures such as arrays or linked lists, where the searching and deleting can take $O(n)$ time in a worst-case scenario. Tree-based structures also maintain the elements in a sorted order. This enables efficient range queries and ordered traversals without the need for extra cost in terms of computational resources. Unlike with Hash tables, which can degrade to $O(n)$ in a collision, a balanced tree can guarantee predictable performance regardless of the input distribution.

2.2.2.2 Disadvantages

Tree-based structures do have certain disadvantages in relation to time complexity and operational overhead. (*Balanced Binary Tree definition & meaning in DSA*, 2025). An unbalanced binary search tree could degrade into a linear structure if the elements inserted are sorted. This causes the core functionality such as search, insertion, and deletion to degrade to $O(n)$ time. (*Binary search tree vs. hash tables*, no date). Even with balanced trees, trying to maintain the height will introduce additional costs. This is because rebalancing functions like rotations must perform during insertions and deletions. This increases the constant-time overhead despite preserving $O(\log n)$ complexity. The deletion functionality can be particularly expensive. They often require multiple adjustments to the structure in order to maintain balance. Tree-based structures also heavily rely on pointer traversal. This reduces the cache efficiency and can result in slower performance compared to that of a hash-based structure.

3 Question 5 - Briefly describe how Question 4 can be updated to support concurrency

In order to support concurrency in Question 4, Hash Table as a LRU Caching, the main aim is to make each `get()` and `put()` method atomic across both shared structures. This being the map or cache and the doubly linked list. As it is now, there are multiple threads which could interleave. This can cause race conditions and also corrupt the map state. (*Synchronization in Java*, 2025). As simple update to the code would be to add a single lock. This lock would go around the critical `get()` and

put() methods using either synchronized methods or blocks or an explicit lock. The explicit lock could be something along the lines of *ReentrantLock* which has more flexible features like a time/try-lock. (*ReentrantLock in Java*, 2025). To have better scalability the *HashMap* could be replaced with a *ConcurrentHashMap*. This would mean map access is thread-safe under contention. You would still however have to lock the list updates because “move-to-front” requires a pointer change which must not interleave. A more common pattern is a *ConcurrentHashMap* with one lock for the list. (*ConcurrentHashMap in Java*, 2025).

4 References

Advantages of BST over Hash Table (2025) *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/dsa/advantages-of-bst-over-hash-table/> (Accessed: November 28, 2025).

Balanced Binary Tree definition & meaning in DSA (2025) *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/dsa/balanced-binary-tree-definition-meaning-in-dsa/> (Accessed: December 18, 2025).

Binary search tree vs. hash tables (no date) *Educative*. Available at: <https://www.educative.io/answers/binary-search-tree-vs-hash-tables> (Accessed: November 28, 2025).

ConcurrentHashMap in Java (2025) *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/java/concurrenthashmap-in-java/> (Accessed: December 27, 2025).

DSA AVL Trees (no date). Available at: https://www.w3schools.com/dsa/dsa_data_avltrees.php?utm_source=copilot.com (Accessed: December 18, 2025).

“Hash Collisions Explained” (no date) *Freeman Law*. Available at: <https://freemanlaw.com/hash-collisions-explained/> (Accessed: November 27, 2025).

Hash Functions and Types of Hash functions - *GeeksforGeeks* (2025). Available at: <https://www.geeksforgeeks.org/dsa/hash-functions-and-list-types-of-hash-functions/> (Accessed: November 26, 2025).

Hash Table Data Structure (2025) *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/dsa/hash-table-data-structure/> (Accessed: November 27, 2025).

Load factor in hash table (no date) *Hyperskill*. Available at: <https://hyperskill.org/learn/step/40135> (Accessed: November 27, 2025).

Open addressing | *Study Glance* (no date). Available at: <http://studyglance.in/php/> (Accessed: November 27, 2025).

ReentrantLock in Java (2025) *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/java/reentrant-lock-in-java/> (Accessed: December 27, 2025).

Synchronization in Java (2025) *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/java/synchronization-in-java/> (Accessed: December 27, 2025).

5 Appendix

GitHub Repo Link: <https://github.com/Oisin003/AADS-Oisin-Gibson-L0017267-CA2.git>

Word Count: 1422

Font: Aptos

Font Size: 12-16